Radar Doppler 24 GHz

Documentation du firmware STM32F767ZI

Explication section par section de `main.c`

Projet L3 EEA

Mars 2026

Ce document couvre l'ensemble du code applicatif du radar, avec le code source commenté et les explications des concepts sous-jacents (FFT, DMA, fenetrage, Doppler).

Table des matières

1 Conventions de ce document **2**

2 Vue d'ensemble de l'architecture **3**
2.1 Tableau récapitulatif des périphériques . . . 3
2.2 Mises à jour globales du projet (STM32 + PC) . . . 3
2.3 Schéma de flux des données . . . 4

3 Includes et constantes **5**
3.1 Fichiers inclus . . . 5
3.2 Constantes de production . . . 5
3.3 Constantes DAC . . . 5

4 Variables globales **7**
4.1 Buffers et flags ADC (production) . . . 7
4.2 Buffers FFT (production) . . . 7
4.3 Debug ADC (production) . . . 8
4.4 Variables DAC . . . 8

5 Fonctions utilitaires **9**
5.1 init_hann_window() – Production . . . 9
5.2 get_sweep_freq() – DAC . . . 9
5.3 update_dac_sweep() – DAC . . . 10

6 Fonction process_fft() – Le coeur du traitement **11**
6.1 Étape 1 : Suppression DC . . . 11
6.2 Étape 2 : Fenêtrage de Hann . . . 11
6.3 Étape 3 : FFT et magnitude . . . 11
6.4 Étape 4 : Recherche du pic . . . 12
6.5 Étape 5 : Estimation du bruit (noise floor) . . . 12
6.6 Étape 6 : Calcul du SNR, décision et sorties . . . 13

7 Fonction main() – Séquence d'initialisation **14**
7.1 Cache, HAL et horloge . . . 14
7.2 Initialisation des périphériques et démarrage . . . 14
7.3 Configuration du DAC . . . 15

8 Boucle principale (while) **17**

9 Callbacks DMA **19**

10 Configuration des périphériques (CubeMX) **20**
10.1 ADC1 – MX_ADC1_Init() . . . 20
10.2 TIM2 – Horloge ADC (reconfigurée à 100 kHz) . . . 20

11 Résumé final **21**

1 Conventions de ce document

Les extraits de code de **production** (nécessaires au fonctionnement du radar) sont présentés sur fond bleu clair :

```
// Exemple de code production
arm_rfft_fast_f32(&fft_instance, fft_in, fft_out, 0);
```

Les extraits de code liés au DAC sont désormais documentés comme partie intégrante du système :

```
// Exemple de code DAC
update_dac_sweep();
```

2 Vue d'ensemble de l'architecture

Le firmware réalise un vélocimètre Doppler en quatre étapes :

1. **Acquisition** : l'ADC1 échantillonne le signal radar à **100 kHz**, piloté par TIM2. Le DMA remplit un buffer circulaire de **2048** mots (interruptions half/full tous les 1024 points).
2. **Accumulation + streaming** : chaque demi-buffer (1024 points) est copié vers `fft_in` et envoyé en **hop UDP** fragmenté vers le PC (port 5555).
3. **Traitement FFT** : suppression DC, fenêtrage de Hann, FFT réelle **4096 points**, recherche de pic sur `[MIN_BIN..MAX_BIN]`, estimation bruit/SNR et conversion Doppler en vitesse.
4. **Sorties** : trame binaire UART3 (compatibilité legacy), mise à jour OLED (5 Hz), LEDs d'état, et surveillance périodique du lien Ethernet.

2.1 Tableau récapitulatif des périphériques

Périphérique	Rôle	Configuration
ADC1	Échantillonnage signal radar	PA0, 12 bits, déclenché par TIM2 TRGO
DMA2 Stream0	Transfert ADC → mémoire	Mode circulaire, half/full callbacks
TIM2	Horloge ADC (runtime)	Prescaler=0, ARR=1079 → 100 kHz (reconfig en USER CODE)
UART3	Sortie résultats / debug	921600 bauds, trames binaires compatibilité legacy
Ethernet RMII	Streaming ADC vers PC	UDP brut, hops de 1024 échantillons, port 5555
OLED SSD1306	Affichage local	I2C1, rafraîchissement 200 ms
DAC1	Génération signal de référence	PA4, DMA1 Stream5 Ch7, TIM4 100 kHz
TIM4	Horloge DAC (runtime)	Prescaler=0, ARR=1079 → 100 kHz

2.2 Mises à jour globales du projet (STM32 + PC)

Depuis la version initiale, le système a évolué de manière significative côté acquisition, traitement et interface :

— **Chaîne STM32 → PC modernisée** : le firmware diffuse les données ADC vers le PC en **hops UDP** (fenêtre glissante), ce qui permet un traitement plus fréquent et une meilleure réactivité qu'un envoi trame complète unique.

— **Analyseur PC `pc_analyzer` accéléré** : FFT glissante avec fréquence de rafraîchissement plus élevée, axe fréquence lisible, curseur de lecture, affichage de vitesse brute et vitesse filtrée.

— **Détection et suivi renforcés** : CFAR réglable en live, lissage spectral, stabilisation du pic, filtre de Kalman configurable, suivi multi-pics (`Top-N tracks`) et métriques de confiance (stabilité SNR, âge de piste, jitter, ratio de pertes).

— **Gestion du bruit et perturbations** : filtrage secteur (50 Hz et harmoniques), plancher de bruit adaptatif, notch configurables (centre/largeur/profondeur), options de calibration automatique (offset DC, normalisation de gain, profil de bruit idle).

— **Fonctions d'exploitation** : mode **record/replay** pour rejouer des captures hors ligne et comparer des réglages A/B.

— **Interface utilisateur remaniée** : menu supérieur `Live Controls` + `Help`, organisation des réglages par contexte (acquisition, CFAR, bruit/notch, tracking, calibration), aides intégrées.

— **Modes de layout** : vues focalisées (`FFT only`, `Scope only`, `Spectrogram only`) et ratios de panneaux réglables.

— **Scope long + échelle verticale par division** : `Scope ms/div` monte jusqu'à **1000 ms/div** avec historique 1 s (**100000 échantillons** à 100 kHz), et réglage `Scope amplitude/div (ADC)` en mode manuel (±4 divisions).

— **Spectrogramme type waterfall** : affichage vertical temporel stabilisé (ancien en haut, récent en bas) pour une lecture plus naturelle de l'évolution Doppler.

— **Portage** `pc_scope` : une partie des améliorations UI/traitement a été reportée dans `pc_scope` (menus, layout, bruit/notch, rendu spectrogramme), avec poursuite de l'alignement fonctionnel avec `pc_analyzer`.

Note : cette documentation reste centrée sur `main.c`. La liste ci-dessus synthétise les évolutions système introduites côté applications PC pour l'exploitation radar temps réel.

2.3 Schéma de flux des données

$$\texttt{PA0 (signal radar)} \xrightarrow{\text{ADC1 @ 100kHz}} \texttt{adc_buf[2048]} \xrightarrow{\text{DMA half/full (1024)}} \texttt{fft_in[4096]} \xrightarrow{\text{FFT}} \texttt{fft_mag[2048]} \xrightarrow{\text{pic/SNR}} \texttt{UDP + UART + OLED}$$

3 Includes et constantes

3.1 Fichiers inclus

```c
#include "main.h"        // Genere par CubeMX : defines des pins,
                         // prototypes HAL, config peripheriques
#include "string.h"      // memcpy, memset (standard C)
#include <math.h>        // powf, sinf, fmodf, log10f
                         // pour le calcul FFT et sweep DAC
#include <stdio.h>       // sprintf pour formater les messages UART
#include "arm_math.h"    // Bibliotheque CMSIS-DSP d'ARM :
                         // fournit les fonctions FFT, multiplication
                         // vectorielle, moyenne, max, cos
                         // optimisees pour Cortex-M7 (SIMD, FPU)
```

Listing 1 – Fichiers d'en-tête inclus

Pourquoi CMSIS-DSP ? La bibliothèque CMSIS-DSP exploite le FPU simple précision et les instructions SIMD du Cortex-M7 pour accélérer les calculs vectoriels. Dans ce projet, elle permet une FFT réelle 4096 points + post-traitement dans la contrainte temps réel ($\sim$ 24 trames/s).

3.2 Constantes de production

```c
#define ADC_BUF_SIZE      2048
// Buffer DMA ADC circulaire (uint16). Half/Full = 1024 echantillons.

#define FFT_SIZE          4096
// FFT reelle embarquee. Resolution: FS/FFT_SIZE = 100000/4096 = 24.41
    Hz/bin.

#define FS                100000.0f
// Frequence d'echantillonnage ADC (runtime via TIM2 ARR=1079).

#define DOPPLER_SCALE     161.0f
// Conversion fd->v pour radar 24.125 GHz.

#define MIN_BIN           2
#define MAX_BIN           250
// Fenetre de recherche pic: ~49 Hz a ~6.1 kHz.

#define SNR_THRESHOLD     15.0f
#define ADC_PP_THRESHOLD  100
// Double seuil de detection: relatif (SNR) + absolu (amplitude ADC).

#define UDP_DST_PORT      5555
#define UDP_SRC_PORT      5555
#define UDP_MAX_PAYLOAD   1400
#define UDP_HOP_SAMPLES   (ADC_BUF_SIZE / 2)  // 1024
// Streaming ADC brut en hops UDP fragmentes.
```

Listing 2 – Définitions des constantes de production (alignées avec main.c)

3.3 Constantes DAC

```c
#define DAC_BUF_SIZE  1024
// Taille du buffer circulaire pour le DAC.
// Le DMA envoie ces 1024 echantillons au DAC en boucle.
// A chaque fin de trame FFT, le buffer est re-rempli
// avec la nouvelle frequence du balayage.

#define SWEEP_F_START 30.0f
// Frequence de debut du balayage en Hz.
// 30 Hz correspond a environ 0.19 m/s en Doppler.

#define SWEEP_F_END   6000.0f
// Frequence de fin du balayage en Hz.
// 6 kHz deborde volontairement de la bande utile (3 kHz)
// pour verifier que MAX_BIN limite bien la detection.

#define SWEEP_TIME_MS 15000
// Duree totale du balayage : 15 secondes.
// Assez lent pour voir clairement les frequences defiler
// sur le terminal UART.
```

Listing 3 – Constantes du générateur de balayage DAC

4 Variables globales

4.1 Buffers et flags ADC (production)

```
volatile uint16_t adc_buf[ADC_BUF_SIZE];
// Buffer circulaire rempli par le DMA depuis l'ADC.
// "volatile" car modifie par le DMA (materiel) en arriere-plan,
// pas par le CPU. Sans volatile, le compilateur pourrait
// optimiser les lectures et lire des valeurs perimees.
// uint16_t car l'ADC est 12 bits (valeurs 0 a 4095).

volatile uint8_t  adc_half_ready = 0;
volatile uint8_t  adc_full_ready = 0;
// Flags positionnes par les callbacks d'interruption DMA.
// adc_half_ready = 1 quand la 1ere moitie (indices 0..1023)
//                    est prete a etre lue
// adc_full_ready = 1 quand la 2eme moitie (indices 1024..2047)
//                    est prete a etre lue
// La boucle principale les remet a 0 apres traitement.
```

Listing 4 – Déclarations des buffers ADC et flags DMA

4.2 Buffers FFT (production)

```
float32_t fft_in[FFT_SIZE];         // 4096 echantillons d'entree
// On y accumule 4 x 1024 echantillons avant de lancer la FFT.
// float32_t = float simple precision (32 bits, ~7 chiffres).
// Les valeurs ADC (0-4095 entiers) sont converties en float
// lors de la copie depuis adc_buf.

float32_t fft_out[FFT_SIZE];        // sortie brute de la FFT
// Contient des nombres complexes entrelaces :
// [re0, im0, re1, im1, re2, im2, ...]
// Donc fft_out[2*k] = partie reelle du bin k
//      fft_out[2*k+1] = partie imaginaire du bin k

float32_t fft_mag[FFT_SIZE / 2];    // 2048 modules (spectre)
// Module = sqrt(re^2 + im^2) de chaque bin.
// fft_mag[k] represente l'amplitude a la frequence k*FS/FFT_SIZE
// Seuls les bins MIN_BIN a MAX_BIN sont examines.

float32_t hann_window[FFT_SIZE];    // coefficients de Hann
// Pre-calcules au demarrage par init_hann_window().
// Multiplies element par element avec fft_in avant la FFT.

arm_rfft_fast_instance_f32 fft_instance;
// Structure opaque de CMSIS-DSP contenant les "twiddle factors"
// (facteurs de rotation) pre-calcules pour une FFT de 4096 pts.
// Initialisee une seule fois par arm_rfft_fast_init_f32().

uint16_t fft_fill = 0;      // compteur : 0 a 4096
// Indique combien d'echantillons ont ete accumules dans fft_in.
// Incremente de 1024 a chaque moitie de buffer DMA.
// Quand il atteint FFT_SIZE (4096), on lance process_fft().
```

Listing 5 – Buffers et état de la FFT

4.3 Debug ADC (production)

```
volatile uint16_t adc_min = 4095;  // valeur min vue
volatile uint16_t adc_max = 0;     // valeur max vue
volatile uint32_t adc_sum = 0;     // somme cumulee
volatile uint32_t adc_count = 0;   // nombre d'echantillons
// Ces statistiques sont collectees pendant la copie des
// echantillons ADC vers fft_in. Elles couvrent une trame
// complete (4096 echantillons).
//
// L'amplitude crete-a-crete (pp) = adc_max - adc_min
// sert de seuil absolu pour la detection :
//   - Signal reel de l'AFE : pp >> 100 LSB
//   - Bruit sur entree flottante : pp ~ 50 LSB
//   - Entree a la masse : pp ~ 10 LSB
//
// Remises a zero apres chaque FFT dans process_fft().
```

Listing 6 – Variables de diagnostic ADC

4.4 Variables DAC

```
uint16_t dac_buf[DAC_BUF_SIZE];
// Buffer circulaire envoye au DAC par DMA.
// Contient 1024 echantillons d'une sinusoide a la
// frequence courante du balayage. Re-rempli a chaque
// trame FFT par update_dac_sweep().

float sweep_phase = 0.0f;
// Accumulateur de phase de la sinusoide DAC.
// Persiste entre les appels a update_dac_sweep() pour
// garantir la CONTINUITE DE PHASE : quand on re-remplit
// le buffer, la sinusoide reprend exactement la ou elle
// s'etait arretee. Sans cela, il y aurait une
// discontinuite qui creerait des harmoniques parasites.

uint32_t sweep_start_tick = 0;
// Timestamp (en ms) du debut du balayage.
// HAL_GetTick() retourne le compteur SysTick (1 kHz).
// Le temps ecoule = HAL_GetTick() - sweep_start_tick
// determine la frequence courante dans le balayage log.
```

Listing 7 – Variables du générateur de signal DAC

5 Fonctions utilitaires

5.1 init_hann_window() – Production

```c
void init_hann_window(void)
{
  for (int i = 0; i < FFT_SIZE; i++)
    hann_window[i] = 0.5f * (1.0f - arm_cos_f32(
                        2.0f * PI * i / (FFT_SIZE - 1)));
}
// Formule : w[i] = 0.5 * (1 - cos(2*pi*i / (N-1)))
//
// POURQUOI LE FENETRAGE ?
// ========================
// La FFT suppose que le signal est periodique sur la
// fenetre d'analyse. Or notre bloc de 4096 echantillons
// est une "coupe" arbitraire du signal continu.
// Les bords du bloc ont generalement des discontinuites
// qui creent des "fuites spectrales" : l'energie d'un
// pic se repand sur les bins voisins.
//
// La fenetre de Hann attenue progressivement les bords
// vers zero :
//   - w[0] = w[N-1] = 0  (bords a zero)
//   - w[N/2] = 1.0        (centre inchange)
//
// Resultat : les fuites spectrales sont reduites d'un
// facteur ~100 (de -13 dB a -32 dB pour le premier
// lobe lateral). Le pic est un peu plus large (2 bins
// au lieu de 1) mais beaucoup plus propre.
//
// arm_cos_f32() utilise une LUT + interpolation,
// plus rapide que cosf() de math.h sur Cortex-M7.
```

Listing 8 – Pré-calcul de la fenêtre de Hann

5.2 get_sweep_freq() – DAC

```c
float get_sweep_freq(void)
{
  uint32_t elapsed = HAL_GetTick() - sweep_start_tick;
  // Temps ecoule en ms depuis le debut du balayage

  if (elapsed >= SWEEP_TIME_MS)
  {
    sweep_start_tick = HAL_GetTick();  // restart
    sweep_phase = 0.0f;
    elapsed = 0;
    // Le balayage a termine ses 15 secondes : on
    // recommence depuis le debut (30 Hz).
    // La phase est remise a zero pour repartir proprement.
  }

  float t_ratio = (float)elapsed / (float)SWEEP_TIME_MS;
  // t_ratio varie lineairement de 0.0 a 1.0
```

```
  return SWEEP_F_START * powf(SWEEP_F_END / SWEEP_F_START,
                              t_ratio);
  // BALAYAGE LOGARITHMIQUE :
  // freq(t) = F_start * (F_end / F_start)^(t/T)
  //
  // A t=0      : freq = 30 * (200)^0      = 30 Hz
  // A t=T/2    : freq = 30 * (200)^0.5    = 424 Hz
  // A t=T      : freq = 30 * (200)^1      = 6000 Hz
  //
  // Pourquoi logarithmique ? Chaque octave (doublement de
  // frequence) recoit le meme temps de balayage.
  // Lineaire donnerait 90% du temps entre 600 et 6000 Hz
  // et seulement 10% entre 30 et 600 Hz.
}
```

Listing 9 – Calcul de la fréquence instantanée du balayage

5.3 update_dac_sweep() – DAC

```
void update_dac_sweep(void)
{
  float freq = get_sweep_freq();
  // Frequence actuelle du balayage (30 a 6000 Hz)

  float phase_inc = 2.0f * PI * freq / FS;
  // Increment de phase par echantillon.
  // A 100 kHz d'echantillonnage et 500 Hz de signal :
  //   phase_inc = 2*pi*500/100000 = 0.0314 rad
  //   soit 100000/500 = 200 echantillons par periode

  for (int i = 0; i < DAC_BUF_SIZE; i++)
  {
    dac_buf[i] = (uint16_t)(2048.0f + 800.0f
                            * sinf(sweep_phase));
    // DAC 12 bits : plage 0 a 4095
    // 2048 = milieu (1.65V sur 3.3V)
    // +/- 800 LSB = +/- 0.64V d'amplitude
    // Signal total : 1.01V a 2.29V

    sweep_phase += phase_inc;
    // L'accumulateur de phase avance d'un pas.
    // Cle de la CONTINUITE DE PHASE : la phase
    // n'est JAMAIS remise a zero entre deux buffers,
    // donc la sinusoide est parfaitement continue.
  }

  if (sweep_phase > 2.0f * PI * 1000.0f)
    sweep_phase = fmodf(sweep_phase, 2.0f * PI);
  // Prevention du debordement float : quand la phase
  // accumule depasse 6283 rad (~1000 tours), on la
  // replie dans [0, 2*pi]. sinf() perd en precision
  // pour les tres grandes valeurs de phase.
}
```

Listing 10 – Remplissage du buffer DAC avec la sinusoïde

6 Fonction process_fft() – Le coeur du traitement

Cette fonction est appelée chaque fois que 4096 échantillons ont été accumulés. Elle constitue le cœur du traitement du signal.

6.1 Étape 1 : Suppression DC

```
void process_fft(void)
{
  float32_t mean;
  arm_mean_f32(fft_in, FFT_SIZE, &mean);
  // Calcule la moyenne arithmetique des 4096 echantillons.
  // Pour un ADC centre a 1.65V : mean ~ 2048 LSB

  for (int i = 0; i < FFT_SIZE; i++)
    fft_in[i] -= mean;
  // Soustrait la moyenne de chaque echantillon.
  // Resultat : le signal oscille autour de 0 au lieu de 2048.
  //
  // POURQUOI ?
  // La composante DC (offset constant) se traduit en un pic
  // enorme au bin 0 de la FFT. Ce pic "deborde" sur les
  // bins voisins (leakage) et masque les petits signaux
  // Doppler. En soustrayant la moyenne, on supprime ce pic.
```

Listing 11 – Suppression de la composante continue

6.2 Étape 2 : Fenêtrage de Hann

```
  arm_mult_f32(fft_in, hann_window, fft_in, FFT_SIZE);
  // Multiplication element par element :
  //   fft_in[i] = fft_in[i] * hann_window[i]
  //
  // arm_mult_f32 est optimisee CMSIS-DSP : elle traite
  // 4 multiplications par cycle grace au pipeline FPU
  // du Cortex-M7. Pour 4096 elements, le cout reste
  // tres faible devant la FFT elle-meme.
  // cycles soit ~2.4 us a 216 MHz.
  //
  // Le resultat est un signal dont les bords valent 0,
  // pret pour la FFT sans fuites spectrales excessives.
```

Listing 12 – Application de la fenêtre de Hann

6.3 Étape 3 : FFT et magnitude

```
  arm_rfft_fast_f32(&fft_instance, fft_in, fft_out, 0);
  // Calcule la FFT reelle 4096 points.
  //
  // "Reelle" signifie que l'entree est purement reelle
  // (pas de partie imaginaire), ce qui permet d'utiliser
  // un algorithme 2x plus rapide que la FFT complexe.
  //
  // Le parametre 0 = FFT directe (1 = FFT inverse).
```

```
//
// Sortie : 4096 floats = 2048 nombres complexes
//   fft_out[0] = DC (partie reelle)
//   fft_out[1] = Nyquist (partie reelle)
//   fft_out[2k], fft_out[2k+1] = re, im du bin k
//
// Complexite : O(N*log2(N)) = 4096*12 = 49152 operations
// Duree typique : ~quelques ms sur Cortex-M7 @216 MHz (selon charge
    systeme)

arm_cmplx_mag_f32(fft_out, fft_mag, FFT_SIZE / 2);
// Pour chaque paire (re, im), calcule :
//   mag = sqrt(re*re + im*im)
//
// Le resultat fft_mag[k] est l'AMPLITUDE au bin k.
// La frequence correspondante est :
//   f = k * FS / FFT_SIZE = k * 24.414 Hz
//
// Exemple : fft_mag[20] = amplitude a 488.3 Hz
```

Listing 13 – FFT rapide réelle et calcul du spectre d'amplitude

6.4 Étape 4 : Recherche du pic

```
float32_t peak_val;
uint32_t peak_idx;
arm_max_f32(&fft_mag[MIN_BIN], MAX_BIN - MIN_BIN,
            &peak_val, &peak_idx);
peak_idx += MIN_BIN;
// Cherche la valeur maximale dans fft_mag[2..249].
// arm_max_f32 retourne la valeur ET l'indice du max.
// On ajoute MIN_BIN car la recherche commence a l'offset
// MIN_BIN, pas a l'indice 0 du tableau.
//
// peak_idx * 24.414 = frequence Doppler detectee
// peak_val = amplitude du pic (en unites lineaires)
```

Listing 14 – Recherche du maximum dans la fenêtre de fréquence

6.5 Étape 5 : Estimation du bruit (noise floor)

```
float32_t noise_sum = 0;
int noise_count = 0;
for (int i = MIN_BIN; i < MAX_BIN; i++)
{
  if (i < (int)peak_idx - 5 || i > (int)peak_idx + 5)
  {
    noise_sum += fft_mag[i];
    noise_count++;
  }
}
float32_t noise_avg = (noise_count > 0) ?
                      noise_sum / noise_count : 1e-12f;
// On calcule la moyenne de TOUS les bins de la fenetre
// SAUF les 11 bins autour du pic (pic +/- 5 bins).
//
```

```
// POURQUOI EXCLURE LE PIC ?
// Si le signal est fort, inclure le pic et ses lobes
// lateraux dans la moyenne du bruit gonflerait
// artificiellement le "bruit", reduisant le SNR calcule.
// En excluant +/- 5 bins, on mesure le vrai bruit de fond.
//
// 1e-12f evite une division par zero si noise_count = 0.
```

Listing 15 – Calcul du plancher de bruit en excluant le pic

6.6 Étape 6 : Calcul du SNR, décision et sorties

```
float32_t peak_db = 20.0f * log10f(peak_val + 1e-12f);
float32_t noise_db = 20.0f * log10f(noise_avg + 1e-12f);
float32_t snr = peak_db - noise_db;

float32_t freq = (float32_t)peak_idx * FS / FFT_SIZE;
float32_t speed_ms = freq / DOPPLER_SCALE;
float32_t speed_kmh = speed_ms * 3.6f;

uint16_t adc_pp = adc_max - adc_min;
adc_min = 4095; adc_max = 0; adc_sum = 0; adc_count = 0;

if (snr > SNR_THRESHOLD && adc_pp > ADC_PP_THRESHOLD)
{
  HAL_GPIO_WritePin(GPIOB, LD1_Pin, GPIO_PIN_SET);
  HAL_GPIO_WritePin(GPIOB, LD3_Pin, GPIO_PIN_RESET);
  last_speed_kmh = speed_kmh;
  last_speed_ms  = speed_ms;
  last_freq      = freq;
  last_snr       = snr;
  last_detected  = 1;
}
else
{
  HAL_GPIO_WritePin(GPIOB, LD1_Pin, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOB, LD3_Pin, GPIO_PIN_SET);
  last_snr      = snr;
  last_detected = 0;
}

// UART binaire legacy:
// 0xAA55 + n_samples + stream_buf[1024]
// 0xBB66 + freq + speed_kmh + speed_ms + snr + det
```

Listing 16 – Décision de détection + sorties UART/OLED/LED

7 Fonction main() – Séquence d'initialisation

7.1 Cache, HAL et horloge

```c
int main(void)
{
  // === CACHE ===
  SCB_EnableICache();
  // Active le cache d'instructions (I-Cache) du Cortex-M7.
  // Accelere l'execution en evitant d'aller chercher chaque
  // instruction en Flash (latence ~7 cycles a 216 MHz).
  //
  // Le D-Cache (donnees) est VOLONTAIREMENT DESACTIVE :
  // le DMA ecrit directement en RAM sans passer par le cache.
  // Si le D-Cache etait actif, le CPU lirait des donnees
  // perimees (cachees) au lieu des donnees DMA fraiches.
  // Solution alternative : invalidation manuelle du cache
  // apres chaque transfert DMA, mais c'est plus complexe.

  HAL_Init();
  // Initialise le HAL, configure SysTick a 1 kHz (1 tick/ms)
  // pour HAL_Delay() et HAL_GetTick().

  SystemClock_Config();
  // Configure la PLL : HSE 8 MHz -> SYSCLK 216 MHz
  // PLL : M=4, N=216, P=2 -> 8/4*216/2 = 216 MHz
  // APB1 = 216/4 = 54 MHz (timers APB1 = 108 MHz)
  // APB2 = 216/2 = 108 MHz (timers APB2 = 216 MHz)
  // Note : les timers sur APB1/APB2 tournent a 2x
  // la frequence du bus si le prescaler bus > 1.
```

Listing 17 – Démarrage du système

7.2 Initialisation des périphériques et démarrage

```c
  // === PERIPHERIQUES (genere par CubeMX) ===
  MX_GPIO_Init();            // LEDs (PB0, PB7, PB14), bouton
  MX_DMA_Init();             // DMA2 Stream0 pour ADC
  MX_ETH_Init();             // Ethernet utilise pour streaming UDP ADC
  MX_USART3_UART_Init();     // UART3 a 115200 (modifie apres)
  MX_USB_OTG_FS_PCD_Init(); // USB (non utilise)
  MX_ADC1_Init();            // ADC1 CH0, TIM2 trigger, 12 bit
  MX_DAC_Init();             // DAC CH1, TIM4 trigger
  MX_TIM2_Init();            // TIM2 CubeMX, puis reconfig runtime a 100
      kHz
  MX_TIM4_Init();            // TIM4 : init CubeMX (modifie apres)

  // === UART RAPIDE ===
  huart3.Init.BaudRate = 921600;
  if (HAL_UART_Init(&huart3) != HAL_OK) Error_Handler();
  // CubeMX initialise UART a 115200 bauds.
  // 115200 est trop lent : une trame FFT toutes les 41 ms
  // avec ~60 caracteres = ~1460 char/s.
  // A 115200 bauds (11520 char/s) ca passerait, mais sans
  // marge pour du debug plus verbeux.
```

```
// 921600 = 8x plus rapide, aucun souci de debit.

// === FFT ===
arm_rfft_fast_init_f32(&fft_instance, FFT_SIZE);
// Pre-calcule les "twiddle factors" pour la FFT 4096 pts.
// Ce sont des valeurs cos/sin utilisees a chaque etage
// de l'algorithme papillon (butterfly) de la FFT.
// Cette init prend ~0.1 ms et n'est faite qu'une fois.

init_hann_window();
// Pre-calcule les 4096 coefficients de la fenetre de Hann.

// === DEMARRAGE ADC + DMA + TIMER ===
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_buf, ADC_BUF_SIZE);
// Lance l'ADC en mode DMA circulaire :
// - Le DMA ecrira dans adc_buf[0..1023] en boucle
// - Interruption a mi-parcours (1024) et a la fin (2048)
// - Le cast (uint32_t*) est requis par le HAL meme si
//   les donnees sont 16 bits (le DMA est configure en
//   half-word dans le MSP)

HAL_TIM_Base_Start(&htim2);
// Demarre TIM2. A partir de cet instant, l'ADC
// echantillonne a 100 kHz. Les donnees arrivent dans
// adc_buf via DMA automatiquement.
```

Listing 18 – Init des périphériques et démarrage de l'acquisition

7.3 Configuration du DAC

```
// === DESACTIVATION IRQ TIM4 ===
HAL_NVIC_DisableIRQ(TIM4_IRQn);
// TIM4 etait utilise pour un toggle LED a 60 Hz.
// On le reutilise pour piloter le DAC a 100 kHz.
// Si l'IRQ restait active, le CPU recevrait 100000
// interruptions/s rien que pour TIM4, ce qui
// saturerait completement le processeur.

sweep_start_tick = HAL_GetTick();
update_dac_sweep();  // remplir le buffer initial

// === CONFIGURATION MANUELLE DMA1 STREAM5 CH7 ===
// Le .ioc ne configure pas le DMA pour le DAC,
// donc on le fait manuellement dans le code utilisateur.
__HAL_RCC_DMA1_CLK_ENABLE();            // horloge DMA1
hdma_dac1.Instance = DMA1_Stream5;       // stream 5
hdma_dac1.Init.Channel = DMA_CHANNEL_7;  // canal 7
// Le mapping stream/channel est fixe par le hardware STM32 :
// DMA1_Stream5_Channel7 = DAC1 (voir Reference Manual table)
hdma_dac1.Init.Direction = DMA_MEMORY_TO_PERIPH;
// Sens : memoire (dac_buf) -> peripherique (registre DAC)
hdma_dac1.Init.PeriphInc = DMA_PINC_DISABLE;
// L'adresse peripherique (DAC->DHR12R1) ne change pas
hdma_dac1.Init.MemInc = DMA_MINC_ENABLE;
// L'adresse memoire avance de +2 octets a chaque transfert
hdma_dac1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
hdma_dac1.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
```

```
28    // 16 bits (half-word) car le DAC est 12 bits aligne a droite
29    hdma_dac1.Init.Mode = DMA_CIRCULAR;
30    // Mode circulaire : quand il arrive a la fin de dac_buf,
31    // le DMA recommence au debut automatiquement.
32    hdma_dac1.Init.Priority = DMA_PRIORITY_LOW;
33    hdma_dac1.Init.FIFOMode = DMA_FIFOMODE_DISABLE;
34    HAL_DMA_Init(&hdma_dac1);
35    __HAL_LINKDMA(&hdac, DMA_Handle1, hdma_dac1);
36    // Lie le handle DMA au handle DAC (requis par le HAL)
37
38    // === TIMER 4 A 100 kHz ===
39    __HAL_TIM_SET_PRESCALER(&htim4, 0);
40    __HAL_TIM_SET_AUTORELOAD(&htim4, 1079);
41    // 108 MHz / (0+1) / (1079+1) = 100000 Hz
42    // Meme frequence que l'ADC pour que le DAC genere
43    // exactement FS echantillons par seconde.
44    HAL_TIM_Base_Start(&htim4);
45
46    HAL_DAC_Start_DMA(&hdac, DAC_CHANNEL_1,
47                      (uint32_t *)dac_buf,
48                      DAC_BUF_SIZE, DAC_ALIGN_12B_R);
49    // Lance le DAC en mode DMA circulaire.
50    // Chaque tick TIM4 (100 kHz), le DMA envoie un
51    // echantillon de dac_buf au DAC.
52    // Le signal sort physiquement sur la pin PA4.
```

Listing 19 – Init manuelle du DMA DAC et démarrage du balayage

8 Boucle principale (while)

```
while (1)
{
  // === PREMIERE MOITIE DU BUFFER (indices 0..1023) ===
  if (adc_half_ready)
  {
    adc_half_ready = 0;  // acquitter le flag

    for (int i = 0; i < ADC_BUF_SIZE / 2; i++)
    {
      uint16_t val = adc_buf[i];   // lire un echantillon
      fft_in[fft_fill + i] = (float32_t)val;  // copier

      // Collecter les stats ADC pendant la copie
      if (val < adc_min) adc_min = val;
      if (val > adc_max) adc_max = val;
      adc_sum += val;
      adc_count++;
    }

    fft_fill += ADC_BUF_SIZE / 2;  // +1024

    if (eth_link_up)
      udp_send_adc_frame(udp_hop_buf, UDP_HOP_SAMPLES, udp_seq_counter
         ++);

    if (fft_fill >= FFT_SIZE)  // 4096 atteint ?
    {
      fft_fill = 0;        // reset pour la prochaine trame
      process_fft();        // lancer le traitement FFT
      update_dac_sweep();  // rafraichir le DAC
      HAL_GPIO_TogglePin(GPIOB, LD2_Pin);  // LED bleue
    }
  }

  // === SECONDE MOITIE DU BUFFER (indices 1024..2047) ===
  if (adc_full_ready)
  {
    adc_full_ready = 0;  // acquitter le flag

    for (int i = 0; i < ADC_BUF_SIZE / 2; i++)
    {
      uint16_t val = adc_buf[ADC_BUF_SIZE / 2 + i];  // +1024
      fft_in[fft_fill + i] = (float32_t)val;

      if (val < adc_min) adc_min = val;
      if (val > adc_max) adc_max = val;
      adc_sum += val;
      adc_count++;
    }

    fft_fill += ADC_BUF_SIZE / 2;

    if (eth_link_up)
      udp_send_adc_frame(udp_hop_buf, UDP_HOP_SAMPLES, udp_seq_counter
         ++);
```

```
54
55        if (fft_fill >= FFT_SIZE)
56        {
57          fft_fill = 0;
58          process_fft();
59          update_dac_sweep();  // rafraichir le DAC
60          HAL_GPIO_TogglePin(GPIOB, LD2_Pin);
61        }
62      }
63
64      // OLED a 5 Hz
65      if (HAL_GetTick() - oled_last_update >= OLED_UPDATE_MS)
66        SSD1306_ShowRadar(last_speed_kmh, last_speed_ms,
67                          last_freq, last_snr, last_detected);
68
69      // Verification lien Ethernet a 1 Hz
70      if (HAL_GetTick() - eth_check_tick >= 1000) { ... }
71    }
72  }
```

Listing 20 – Pattern double-buffer dans la boucle principale

Explication du pattern double-buffer

Le DMA remplit `adc_buf[2048]` en continu et de manière circulaire. Le CPU et le DMA ne doivent jamais accéder à la même moitié en même temps :

Phase	DMA écrit dans	CPU lit
Half-transfer	indices 1024..2047	indices 0..1023
Full-transfer	indices 0..1023	indices 1024..2047

Cadences principales :

- Hop DMA/UDP : $1024/100000 = 10.24\,ms$ (~ 97.6 hops/s)
- Trame FFT : $4096/100000 = 40.96\,ms$ (~ 24.4 FFT/s)
- OLED : 5 Hz ; vérification lien Ethernet : 1 Hz

9 Callbacks DMA

```c
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
  if (hadc->Instance == ADC1) adc_half_ready = 1;
}
// Appelee par le HAL depuis l'ISR DMA2_Stream0.
// Declenchee quand le DMA a rempli adc_buf[0..1023].
// Ne fait QU'une seule chose : lever un flag.
//
// REGLE D'OR DES ISR :
// Les routines d'interruption doivent etre les plus courtes
// possible. Le traitement lourd (FFT, UART) se fait dans
// la boucle principale, pas dans l'ISR.

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
  if (hadc->Instance == ADC1) adc_full_ready = 1;
}
// Idem, mais declenchee quand adc_buf[1024..2047] est rempli.
// Le DMA recommence ensuite a l'indice 0 (mode circulaire).

/* TIM4 callback desactive :
 * TIM4 pilote maintenant le DAC DMA a 100 kHz.
 * L'ancien callback (toggle LED a 60 Hz) a ete retire.
 * Si l'IRQ TIM4 etait active, 100000 interruptions/s
 * consommeraient ~30% du CPU pour rien.
 */
```

Listing 21 – Callbacks d'interruption DMA pour l'ADC

10 Configuration des périphériques (CubeMX)

10.1 ADC1 – MX_ADC1_Init()

```
hadc1.Instance = ADC1;
hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
// Horloge ADC = APB2 / 4 = 108 / 4 = 27 MHz
// Frequence max de l'ADC STM32F7 = 36 MHz, on est ok.

hadc1.Init.Resolution = ADC_RESOLUTION_12B;
// 12 bits : valeurs de 0 a 4095

hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
hadc1.Init.ContinuousConvMode = DISABLE;
// Un seul canal, pas de scan. Le mode continu est
// desactive car c'est TIM2 qui declenche chaque conversion.

hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_RISING;
hadc1.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T2_TRGO;
// Declenchement sur front montant du TRGO de TIM2.
// A chaque evenement Update de TIM2 (runtime: 100 kHz), une
// conversion ADC demarre automatiquement.

hadc1.Init.DMAContinuousRequests = ENABLE;
// Le DMA continue de transferer apres chaque conversion
// (mode circulaire). Sans ca, le DMA s'arreterait apres
// le premier buffer.

// Canal echantillonne :
sConfig.Channel = ADC_CHANNEL_0;  // PA0
sConfig.SamplingTime = ADC_SAMPLETIME_15CYCLES;
// Temps de conversion total =
//   (15 cycles sampling + 12 cycles conversion) / 27 MHz
//   = 27 / 27e6 = 1.0 us (ordre de grandeur)
// A 100 kHz (10 us entre chaque conversion), largement ok.
```

Listing 22 – Configuration de l'ADC1

10.2 TIM2 – Horloge ADC (reconfigurée à 100 kHz)

```
htim2.Instance = TIM2;
htim2.Init.Prescaler = 0;
htim2.Init.Period = 2159;  // ARR = 2159
// Valeur CubeMX (50 kHz). Dans USER CODE, ARR est force a 1079
// pour obtenir 100 kHz runtime :
//   f = 108e6 / (PSC+1) / (ARR+1) = 108e6 / 1 / 1080 = 100 kHz.

sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
// L'evenement Update est route vers la sortie TRGO.
// Le TRGO de TIM2 est connecte a l'entree trigger
// de l'ADC1 (T2_TRGO), ce qui declenche une conversion.
```

Listing 23 – Configuration du timer d'échantillonnage

11 Résumé final

Le firmware actuel combine :

- acquisition ADC à 100 kHz + DMA circulaire,
- traitement FFT 4096 points avec détection Doppler,
- streaming UDP hops vers le PC (port 5555),
- télémétrie UART legacy et affichage OLED,
- génération DAC synchronisée (TIM4 + DMA) conservée dans l'architecture.

Fin du document – Projet L3 EEA Radar Doppler 24 GHz